Robust Binary Audio Classification with SNNs

Eashan Patel
Student at Rutgers University
New Brunswick, New Jersey, USA
emp179@scarletmail.rutgers.edu

Krishaan Chaudhary
Student at Rutgers University
New Brunswick, New Jersey, USA
kac551@scarletmail.rutgers.edu

***Abstract*—Spiking neural networks (SNNs) are an alternative to conventional deep learning methods like artificial neural networks (ANNs) for temporal sensory data, with the potential for more brain-like and energy-efficient processing. We study a specific setting: binary classification of one-second spoken words from the Google Speech Commands dataset. Audio is converted into event-based spike trains using the Speech2Spikes encoding pipeline, which represents changes in spectral energy across 20 mel-frequency bands over 200 time steps as positive/negative spikes. We train a fully-connected SNN with leaky integrate-and-fire (LIF) neurons using surrogate gradients and a rate-based mean-square-error objective that encourages the correct output neuron to fire at a high rate. To evaluate robustness, we introduce two distortions, white noise and random pitch shifting, and compare (i) training only on clean data versus (ii) training on a mixture of clean and distorted data. Clean-only training achieves high accuracy on clean test audio (95.8%) but degrades on distorted audio (61.6% for white noise; 63.1% for pitch). Mixing distorted examples into training largely restores performance on distorted tests (93.7% for white noise; 88.0% for pitch) with minimal loss on clean data (95.4% and 94.3%, respectively). We also compare the efficacy of these SNN classifiers with baseline ANN classifiers under similar parameters for model architecture and data representation.**

***Index Terms*—spiking neural networks, keyword spotting, speech recognition, robustness, surrogate gradients, data augmentation, neuromorphic encoding.**

I. Introduction

The computational problem addressed in this project is robust keyword classification: given a short speech waveform, the goal is for the model to predict which word was spoken. A practical keyword spotter must operate in noisy and unpredictable environments; for example, a voice assistant listening in a room with background chatter or music. Humans handle these conditions well but in contrast, machine learning models struggle when test audio differs from the training distribution. Conventional speech recognition systems typically rely on deep neural networks and transformer-style architectures that deliver high accuracy. The downside here is that these models can be memory-intensive for always-on devices. SNNs offer a neuro-inspired alternative that processes information through sparse spikes and temporal dynamics. The main question we ask is: can an SNN paired with an auditory spike encoder achieve strong accuracy on a real speech dataset and remain robust to noise and pitch distortions?

II. Motivation and Context

Our project pitch was inspired by the brain's ability to interpret speech in noisy settings and the limitations of existing SNN approaches, including performance loss from ANN-to-SNN conversion and limited robustness to noise. More broadly, speech recognition with SNNs is difficult because the input is temporal and high-dimensional, so converting raw waveforms into informative spike trains is nontrivial. Speech also varies across speakers (accent, pitch, speaking rate) and recording conditions (background noise, poor microphone quality), making generalization difficult. Rather than aiming immediately for large-vocabulary recognition, we focus on a controlled binary setting that still captures real-world challenges: two-word classification with explicit distortions. This lets us evaluate end-to-end behavior and directly test whether data augmentation in spike space improves generalization under distribution shift.

III. Steps

A. Dataset and Task Formulation

We use one-second spoken-word clips from the Google Speech Commands dataset [1]. Each audio file is loaded as a waveform and then converted into spikes. The frequency is 16 kHz. To construct a binary task, we select two classes (e.g., "cat" vs. "dog"). In our preprocessing pipeline, we sample 500 clips per class and split them into training and test sets using an 80/20 ratio. This produces balanced training and test sets while keeping runtime manageable.

B. Distortion and Data Augmentation

We create two distorted datasets to simulate common audio shifts. First, we apply additive Gaussian white noise by sampling noise with mean 0 and standard deviation equal to the signal's standard deviation, then scaling it by a noise-percentage factor. Second, we apply random pitch shifting in semitone steps using a standard audio effect while keeping the clip length fixed. Distorted clips are written to parallel folder structures (one per word class), enabling the same preprocessing code to build spike datasets for clean and distorted audio. These distortions represent two different failure modes: white noise primarily lowers signal-to-noise ratio across all frequencies, while pitch shifting can change spectral energy allocation across mel bands. Both directly affect the features that spike encoders typically rely on.

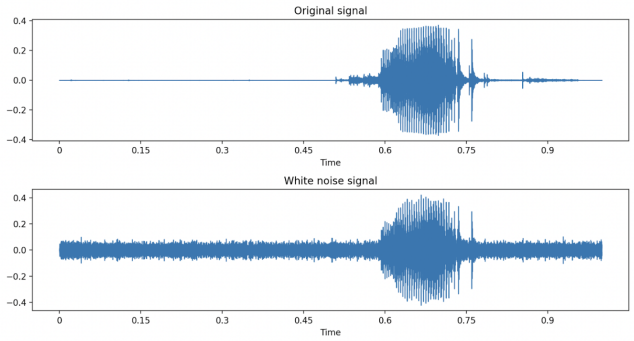

Fig. 1. This figure shows how a .wav file changes under a Gaussian white noise transformation. Original audio wav (above), transformed (below).

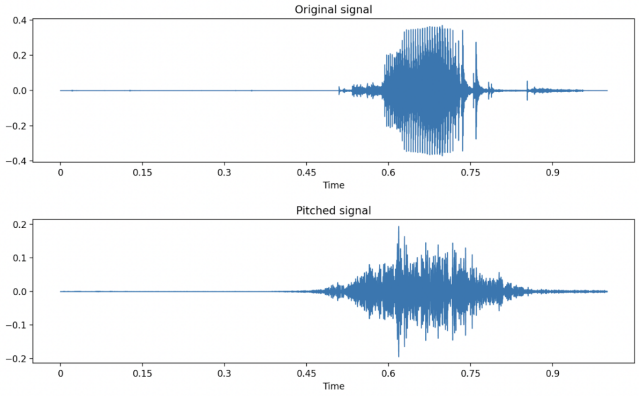

Fig. 2. This figure shows how a .wav file changes under a pitch transformation. Original audio wav (above), transformed (below).

C. Speech2Spikes Encoding

Raw audio is converted to a spiking representation using the Speech2Spikes (S2S) [2] encoding framework. The encoding process: (1) centers and scales the waveform for normalization; (2) computes a sliding discrete Fourier transform across 200 time intervals; (3) aggregates energy into 20 mel-frequency bands (a derived property of audio signals that groups ranges of frequencies into buckets and has been shown to be effective for audio classification [3]); and (4) emits spikes based on sharp energy changes per band. Two neurons are assigned to each band: one for positive (increasing energy) events and one for negative (decreasing energy) events. The output is a 2×20×200 tensor of binary spikes.

Implementation-wise, we store the spike events: for each clip, we record indices (neuron, time) where positive or negative spikes occur. During training and evaluation, we reconstruct a tensor with two channels (positive/negative). For some distorted encodings, the maximum time index produced by the encoder can vary; to keep input shapes fixed, we rescale time indices so the largest event aligns with the final time bin (199). This preserves relative timing while ensuring compatibility with a fixed-length SNN simulation.

While setting up the training dataset for the ANN baseline model, we transform the input to 20 firing rates for each of the 2 channels as the analog for the integrate and fire behavior of the LIF Neurons in the SNN.

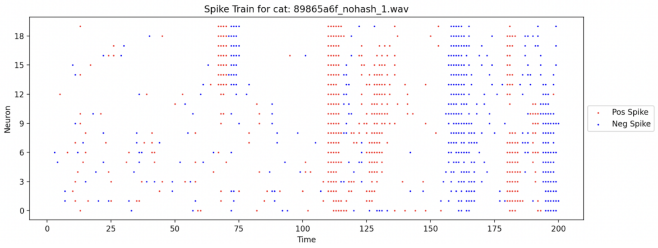

Fig. 3. Above is a sample encoding for a processed .wav file with a 1 second clip of cat.

D. Spiking Network Architecture

We implement a feedforward SNN in PyTorch using snnTorch Leaky Integrate-and-Fire (LIF) neurons. At each time step t, the input slice $x_t \in \{0,1\}^{\{2\cdot 20\}}$ is flattened to 40 features and passed through four fully-connected layers: 40→256→256→256→2. Each hidden layer uses LIF dynamics, and the output layer also uses LIF to produce spikes for two class neurons. The LIF neuron maintains a membrane potential (state) that decays over time and integrates incoming current.

In discrete time, a typical update is $u_{t+1} = \beta u_t + I_t - s_t u_{reset}$, where:

- β controls leak/decay
- I_t is synaptic input
- s_t is the output spike

When u_t crosses a threshold, a spike is emitted and the state is partially reset. In our code, we fix β = 0.95 and simulate T = 200 time steps per sample. Because spikes are non-differentiable, we use a fast-sigmoid surrogate function [4] during backpropagation. Following is a visual depiction of the SNN Network Architecture.

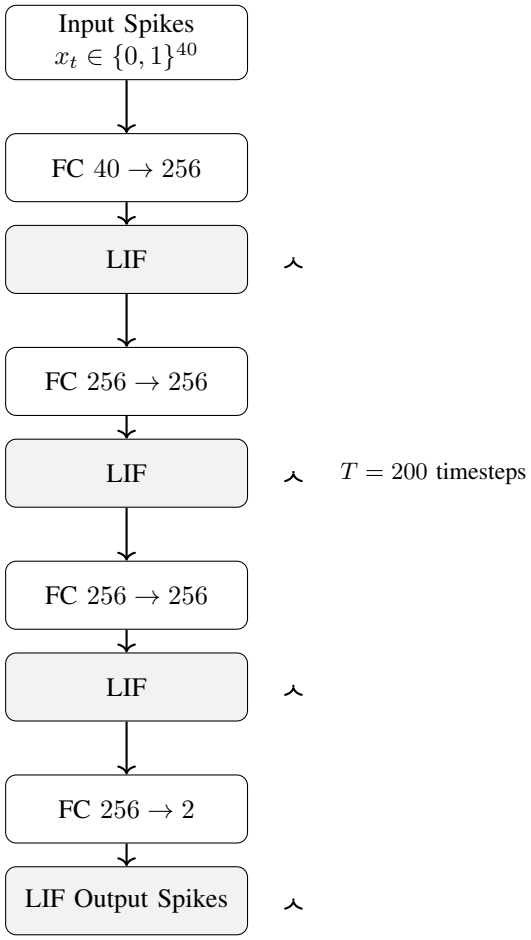

Fig. 4. Feedforward SNN with LIF neurons. Dynamics repeated for $T = 200$ timesteps with leak factor $\beta = 0.95$.

E. Baseline Artificial Neural Network (ANN) Architecture

For the baseline ANN, we also use 4 fully connected layers: 40→256→256→256→2. The input spikes are parsed into firing rates. The key difference is here that there are no LIF neuron layers, and we use Rectified Linear Unit (ReLU) in between the layers instead to capture nonlinearity.

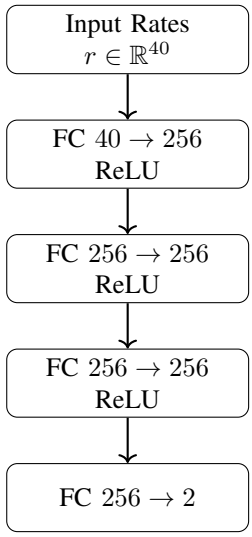

Fig. 5. Baseline ANN architecture. Input spike trains are converted to firing rates and processed through fully connected layers with ReLU nonlinearities.

F. Training Objective and Optimization

We use a rate-based objective at the output layer. For each sample, we define a target firing rate vector r with $r_y = r_{high}$ and $r_{\neg y} = r_{low}$, where $r_{high} = \frac{200}{1000}$ and $r_{low} = \frac{20}{1000}$. Given an output spike train s_t over T steps, we compute empirical firing rates $r = \frac{1}{T}\sum_{t=1}^{T} s_t$ and minimize Mean-square Error. This training scheme encourages the correct class neuron to fire frequently while suppressing the other neuron. We optimize the network using Adam (learning rate 5×10▫4) for 40 epochs. For prediction, we sum output spikes across time and select the neuron with the largest spike count. This matches the intuition of rate coding and reduces sensitivity to small timing variations.

For the baseline ANN, we pass in 2 channels of 20 firing rates as input. The number of epochs, learning rate and optimizer are kept the same as the SNN parameters.

G. Implementation and Reproducibility

Our codebase separates the pipeline into three stages: (1) preprocessing scripts that generate spike datasets from audio folders (clean, white-noise, and pitched), (2) training scripts that fit SNN weights under different training regimes, and (3) evaluation scripts that load trained weights and report accuracy on selected spike datasets. Spike datasets are stored as NumPy arrays in dedicated output directories (e.g., processed_spike_data/ for clean, wn_processed_spike_data/ for white noise, and pitched_processed_spike_data/ for pitch). This structure makes it straightforward to swap word pairs or add additional distortions. We have a similar setup for the baseline ANN training and testing procedures.

IV. EXPERIMENTAL DESIGN

We evaluate robustness under two cases for each distortion type:

Case 1 (Clean-only training). Train the SNN on clean spike data only. Evaluate on both clean test spikes and distorted test spikes.

Case 2 (Mixed training). Train the SNN on a union of clean and distorted spike data. Evaluate on both clean and distorted test spikes.

We repeat this process with the baseline ANN for a control.

V. RESULTS

Table I summarizes test accuracy for both distortion types. The clean-only model performs strongly on clean audio but fails to generalize to distortions. Training on the mixture substantially improves distorted-test accuracy while preserving near-baseline performance on clean data.

Table II shows the baseline accuracy under the corresponding test cases for an ANN.

Tables III and IV highlight key model and training hyperparameters.

TABLE I
SNN ACCURACY BY TRAINING REGIME AND DISTORTION TYPE.

Distortion	Train Data	Test: Clean	Test: Distorted
White Noise	Clean only	95.8%	61.6%
White Noise	Clean + Distorted	95.4%	93.7%
Pitch Change	Clean only	95.8%	63.1%
Pitch Change	Clean + Distorted	94.3%	88.0%

TABLE II
ANN ACCURACY BY TRAINING REGIME AND DISTORTION TYPE.

Distortion	Train Data	Test: Clean	Test: Distorted
White Noise	Clean only	73.3%	51.6%
White Noise	Clean + Distorted	74.3%	80.6%
Pitch Change	Clean only	73.3%	60.9%
Pitch Change	Clean + Distorted	71.1%	67.4%

TABLE III
SNN - KEY MODEL AND TRAINING HYPERPARAMETERS

Parameter	Value
Input neurons	2×20 (pos/neg × mel bands)
Timesteps	200
Hidden layers	3 layers × 256 LIF neurons
Output neurons	2 LIF neurons
Membrane delay	0.95
Surrogate gradient	fast sigmoid
Optimizer	Adam, lr = $5 \cdot 10^{-4}$
Epochs	40
Target rates	$r_{high} = 0.2, r_{low} = 0.02$

Two trends are clear. First, distortion causes a large domain shift in spike space: clean-only training drops from 95.8% on clean audio to around 62% on distorted tests. Second, mixed training acts as a form of spike-domain data augmentation that dramatically improves robustness: white-noise accuracy rises to 93.7% and pitch accuracy rises to 88%, while clean accuracy remains above 94%. The remaining gap for pitch suggests that pitch shifting is a harder task for our encoder/model combination, likely because shifting redistributes energy across mel bands and can change the timing of positive/negative energy transitions.

TABLE IV
ANN - KEY MODEL AND TRAINING HYPERPARAMETERS

Parameter	Value
Input neurons	2×20 (pos/neg × firing rates)
Hidden layers	3 layers × 256 Neurons w/ ReLU
Output neurons	2 neurons
Optimizer	Adam, lr = $5 \cdot 10^{-4}$
Epochs	40
Target rates	$r_{high} = 0.2, r_{low} = 0.02$

In the case of the ANN, the overall accuracy is considerably lower across all the cases, indicating the importance of the time dynamics that the SNN is better able to capture. We see it lag behind significantly, with accuracy on the baseline case of 73%. There is a significant dropoff when trying to extrapolate a model trained only on clean data to a model also trained on distorted data. When there is distorted data added to the training set, the dropoff is not as stark.

VI. DISCUSSION

Our results support the core intuition behind the project: even with a relatively simple feedforward SNN, event-based auditory representations can be used to achieve high accuracy on real speech data, but robustness depends strongly on training distribution. Our results show that a SNN outperforms an ANN with comparable architecture, but it still has limitations. The main weaknesses we observed matches a common weakness in prior speech-recognition pipelines: models trained on a single "clean" distribution can work well with the acoustic patterns present during training but degrade under domain shift. In our setting, white noise and pitch shifting both alter the spectral energy trajectory used by the Speech2Spikes encoder to emit events. When trained only on clean spikes, the SNN appears to learn decision boundaries that rely on fine-grained spike statistics that are not stable under these transformations. Mixed training exposes the network to a broader set of spike patterns and encourages it to learn more invariant mappings. From a neuro-inspired perspective, the pipeline has two key "brain-like" components: (i) an event-based encoding that emphasizes changes in frequency-band energy (similar to auditory nerve firing driven by stimulus changes), and (ii) a dynamical network of LIF neurons that integrates information over time. While our training uses surrogate gradients, it enables efficient optimization and strong performance, and it provides a baseline that future work could compare to local learning rules such as STDP, which is more rooted in bioogical plausbility [5]. There are also important limitations. We focused on binary classification and a fully connected architecture, so scaling to large vocabularies may require more structured models (e.g., convolutional or recurrent SNNs) and more extensive augmentation. We also did not measure energy consumption or latency, so we cannot directly quantify neuromorphic efficiency. Finally, we tested only two

distortions; additional modifications like reverberation, time-stretching, and background speech could be included to better approximate real environments.

VII. CONCLUSION AND FUTURE WORK

We presented an SNN-based approach for robust binary audio classification using Speech2Spikes encoding. A compact LIF network trained with a rate-based surrogate-gradient objective achieves strong performance on clean audio samples and, with appropriate data augmentation, maintains high accuracy under white noise and pitch distortions. Future work includes extending to multi-class vocabularies, scanning distortion strength to locate human-versus-model breakdown points, measuring energy/latency on neuromorphic hardware or simulators, and exploring alternative learning rules (e.g., STDP) or hybrid training to improve biological plausibility while maintaining accuracy.

REFERENCES

[1] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018.

[2] K. M. Stewart, T. M. Shea, N. Pacik-Nelson, E. Gallo, and A. Danielescu, "Speech2spikes: Efficient audio encoding pipeline for real-time neuromorphic systems," in *Proceedings of the 2023 Annual Neuro-Inspired Computational Elements Conference (NICE '23)*, pp. 71–78, Association for Computing Machinery, 2023.

[3] M. Ashames and S. Ergin, "Mel-spectrograms and data augmentation for spoken digit classification," in *Proceedings of the International Conference on Advances in Engineering, Architecture, Science and Technology*, dec 2021.

[4] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 61–63, 2019.

[5] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, "Deep learning in spiking neural networks," *Neural Networks*, vol. 111, p. 47–63, Mar. 2019.